

LG CNS • 2026 ARCHITECTURE DEEP DIVE — AGENTIC RAG WORKSHOP

기업 데이터 연동을 위한 에이전틱 RAG 아키텍처 구축 실무

엔터프라이즈 데이터 인덱싱 · MCP & Skills 다중 연동 · 평가 기반 자가 개선 (Self-Correction)

INSTRUCTOR

김형욱 강사

CONTACT

hukim@artiasolution.com

COURSE LEVEL

Production Practical Workshop (v1.3)

강의 목차 (Table of Contents)

Module 01. 에이전틱 RAG 아키텍처 개요 (11 Slides)

Page 3

Module 02. 엔터프라이즈 데이터 인덱싱 (19 Slides)

Page 14

Module 03. MCP & Skills: 다중 데이터 소스 연동 (18 Slides)

Page 33

Module 04. 평가 기반 자가 개선 (Self-Correction) (18 Slides)

Page 51

DAY 1 · MODULE 1

에이전틱 RAG 아키텍처 개요

검색을 '파이프라인의 단계'에서 '에이전트의 도구'로

MODULE OVERVIEW

과정 소개 및 1교시 학습 목표

• RAG의 발전 흐름과 전환 이유

- Naive → Advanced → Modular → Agentic RAG, 무엇이 바뀌었고 왜 바뀌었는가

• Agentic RAG 아키텍처 (RAG as a Tool)

- 검색을 파이프라인의 한 단계가 아니라, 에이전트가 필요할 때 호출하는 도구로 설계

• 구현을 위해 알아야 할 것

- 에이전트 루프, 도구 인터페이스, 데이터 계층, 연결 표준(MCP), 평가

• RAG는 죽었는가: 검색의 확장과 진화

- "RAG is dead" 주장의 실체와, 검색이 에이전트의 기본 능력으로 확장되는 방향

EVOLUTION

RAG의 발전: 누가 검색 흐름을 결정하는가

Naive RAG

2020 ~ 2023

개발자가 설계한 것
검색 1회 → 생성흐름 결정
개발자 (고정)

대표 기법

청킹 임베딩 Top-k

한계

복합 질문, 여러 소스에 걸친 질문에 약함

2020 RAG 논문 (Lewis et al.)



Advanced RAG

2023 ~ 2024

개발자가 설계한 것
검색 전·후 처리흐름 결정
개발자 (고정)

대표 기법

질의 재작성 HyDE 하이브리드 검색

Rerank

한계

처리 단계는 늘었지만 흐름은 여전히 고정

2023.12 RAG 서베이 (Gao et al.)
Naive / Advanced / Modular 분류

Modular RAG

2024

개발자가 설계한 것
모듈 조합과 라우팅 규칙흐름 결정
개발자 (규칙 기반 분기)

대표 기법

라우터 반복 검색 모듈 교체

한계

질문 유형이 늘수록 분기 규칙 관리 부담

2024.07 Modular RAG (Gao et al.)



Agentic RAG

2025 ~

개발자가 설계한 것
도구와 데이터흐름 결정
모델 (실행 중 판단)

대표 기법

ReAct Tool Calling MCP

과제

비용 · 지연 증가 → 평가로 관리

2024.11 MCP 공개
2025.01 Agentic RAG 서베이 (Singh et al.)기존의 RAG가 사라진 것이 아니라 **에이전트의 검색 도구로 들어갔습니다**. RAG의 핵심 아이디어와 기법들은 여전히 활용되고 있습니다.

PROBLEM DIAGNOSIS

Naive RAG의 4가지 한계

1. 단발 검색 (Single-shot Retrieval)

질문 하나에 검색 한 번으로 끝납니다. 앞에서 찾은 내용을 보고 다음에 무엇을 찾을지 정할 수 없어, 여러 단계를 거쳐야 하는 질문에 약합니다.

비유 검색창에 한 번 입력하고, 첫 페이지 결과만 읽고 보고서를 쓰는 것

2. 단일 소스 (Single Source)

미리 연결된 저장소 하나에서만 찾습니다. 답이 다른 곳에 있거나 여러 곳에 나뉘어 있으면 처음부터 찾을 방법이 없습니다.

비유 책장 하나만 있는 도서관에서 모든 질문의 답을 찾으려는 것

3. 표현의 차이 (Query-Document Mismatch)

사용자의 질문 문장을 그대로 검색에 씁니다. 질문과 문서가 같은 내용을 다른 말로 표현하면 관련 문서를 놓치거나 엉뚱한 문서를 가져옵니다.

비유 "노트북이 안 켜져요"로 찾는데 매뉴얼에는 "전원 인가 불량"이라고 적혀 있는 것

4. 판단 부재 (No Judgment)

가져온 내용이 답하기에 충분한지 확인하지 않고 바로 답을 만듭니다. 근거가 부족해도 그럴듯한 답이 나옵니다 (환각).

비유 펼친 페이지에 답이 없는데도 그 페이지를 근거로 답안을 채우는 것

WHY NOW

왜 Agentic RAG로 바뀌었나

① 모델이 스스로 판단할 수 있게 되었다

도구 호출(Tool Calling)과 여러 단계의 추론이 안정되면서, "검색할지, 어디서, 몇 번, 언제 멈출지"를 모델에 맡길 수 있게 되었습니다.

비유 시키는 대로만 찾던 신입이, 무엇을 찾아볼지 스스로 정하는 경력자가 된 것

② 질문이 복잡해졌다

실제 업무 질문은 비교하거나, 여러 단계를 거치거나, 여러 곳의 자료를 합쳐야 답할 수 있습니다. 한 번의 검색으로는 필요한 근거를 모두 모을 수 없습니다.

비유 여행 계획: 항공권, 날씨, 숙소를 각각 찾아본 뒤 조합해야 답이 나옴

③ 규칙으로는 감당할 수 없었다

질문 유형마다 분기 규칙(if/else)을 추가하는 방식은 유형이 늘수록 관리할 수 없게 됩니다. 흐름 판단은 모델에게 맡기고, 사람은 도구와 데이터 설계에 집중합니다.

비유 모든 상황을 매뉴얼로 정해 둔 상담원 vs 원칙과 도구를 받고 판단하는 상담원

▲ 대가: 비용과 지연

여러 번 찾아보는 만큼 시간과 비용이 듭니다. 단일 검색 대비 토큰은 3~10배, 지연은 2~5배까지 늘어납니다. 그래서 개선 효과를 평가로 확인해야 합니다.

업계 보고 응답 시간 1~2초(단일 검색) → 8~12초(에이전트 루프)

PARADIGM SHIFT

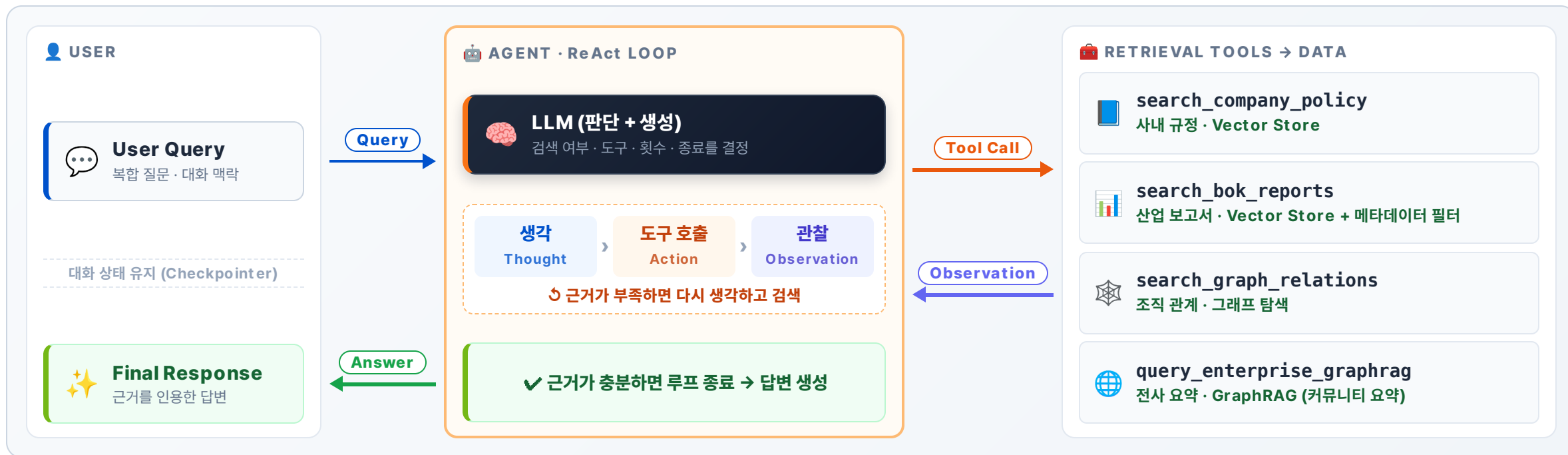
파이프라인 RAG vs Agentic RAG 비교

비교 항목	Naive · Advanced RAG (파이프라인)	Agentic RAG (도구)
핵심 관점	검색은 정해진 파이프라인의 한 단계	검색은 에이전트가 필요할 때 호출하는 도구
흐름 결정	개발자가 설계 시점에 고정	모델이 실행 중에 판단 (검색 여부 · 소스 · 횟수 · 종료)
검색 횟수	1회 (고정)	0회 ~ N회 , 결과를 보고 다음 검색을 정함
검색 소스	주로 단일 Vector Store	여러 검색 소스를 도구로 노출 (Vector · 키워드 · 그래프 · MCP)
질의 처리	질의 재작성 · HyDE 등을 파이프라인에 미리 배치	모델이 질문을 나눠서 여러 번 검색 (예: 기간별로 나눠 검색)
비용 · 지연	낮고 예측 가능	높고 질문마다 다름 → 평가로 관리

Advanced RAG의 기법(재작성, 하이브리드, Rerank)은 여전히 유효합니다. 차이는 **그 기법을 언제, 몇 번 쓸지를 누가 정하느냐**입니다.

RETRIEVAL AGENT ARCHITECTURE

Retrieval Tool 기반 에이전틱 RAG 아키텍처



● 단일 소스 → 여러 도구

저장소마다 검색 도구를 만들어 노출하고, 모델이 질문에 맞는 도구를 고릅니다.

● 단발 검색 → 반복 호출

앞 도구의 결과를 보고 다음 검색어와 도구를 정합니다. 필요하면 질문을 나눠 여러 번 호출합니다.

● 판단 부재 → 모델의 판단

결과가 충분한지 보고 검색을 멈추거나 계속합니다. 대화 상태는 체크포인트로 유지합니다.

RETRIEVAL OPTIONS

검색 소스의 선택지: 데이터와 질문에 맞게 고른다

데이터 · 질문의 성격	검색 방식	대표 도구 · 사례
의미가 비슷한 문장 찾기	Vector 검색 (Dense Embedding)	Chroma · pgvector · Qdrant · Milvus
조항 번호 · 고유명사 · 정확한 용어	키워드 검색 (BM25 · 전문 검색)	Elasticsearch · OpenSearch · SQLite FTS
사람 · 조직 · 관계 따라가기	지식 그래프 · GraphRAG	Neo4j · NetworkX · Microsoft GraphRAG
수치 · 집계 등 정형 데이터	SQL (Text2SQL)	PostgreSQL · 사내 DW
구조화된 문서 직접 탐색	파일 탐색 (glob · read · grep)	Claude Code · Cursor 등 코딩 에이전트
사내 시스템 · 이종 소스 연결	MCP 서버	FastMCP · 사내 API · SaaS 커넥터

정답 저장소는 하나가 아닙니다. **데이터와 질문에 맞게 고르고, 고른 것을 도구로 노출합니다.** 하나의 에이전트가 여러 검색 방식을 함께 쓰는 하이브리드 구성이 일반적입니다.

INSIGHT · RAG IS DEAD?

RAG는 죽었는가? 죽은 것은 '고정 파이프라인'이다

자주 듣는 주장

"컨텍스트가 길어졌으니 문서를 통째로 넣으면 된다"

"코딩 에이전트는 Vector DB 없이 grep만으로 잘 찾는다"

"에이전트가 파일을 직접 열어 읽으면 DB는 필요 없다"

실제로 일어난 일



책 몇 권은 넣을 수 있어도 회사 전체 문서는 넣을 수 없습니다. 많이 넣을수록 비싸지고, 모델은 핵심을 놓칩니다. 결국 **무엇을 넣을지 고르는 일, 즉 검색**이 필요합니다.



코드는 함수명 · 파일명처럼 **정확한 단어**로 찾는 데이터라 grep이 잘 맞습니다. 하지만 같은 뜻을 다른 말로 묻는 질문에는 **Vector 검색**이, "누가 누구와 연결되어 있나"에는 **Graph**가 필요합니다.



문서가 수십 개라면 가능합니다. 수만 개라면 하나씩 열어 볼 수 없습니다. **Vector DB는 어디부터 볼지 후보를 좁히고, Graph DB는 흩어진 관계를 미리 이어 둡니다.** 파일 탐색과 DB는 경쟁이 아니라 역할 분담입니다.

사라지고 있는 것은 "한 번 검색하고 끝"이라는 **고정 파이프라인**입니다. Vector DB · Graph DB · 파일 탐색은 모두 **에이전트가 골라 쓰는 검색 도구**로 남아, 데이터의 성격에 맞게 함께 쓰입니다.

INSIGHT · WHERE RAG IS GOING

RAG는 어떻게 확장되고 있는가: 4가지 방향

01 · 검색 방식

하나의 벡터 DB에서
검색 포트폴리오로Vector 단일 → Vector · 키워드 · 그래프
· SQL · 파일 탐색

Vector DB는 여전히 의미 검색의 중심입니다. 여기에 키워드 · 그래프 · 파일 탐색이 더해져, 에이전트가 질문에 맞게 조합합니다.

코딩 에이전트의 grep 기반 탐색, 키워드 검색 에이전트 연구

02 · 지식 구조

잘린 청크에서
탐색 가능한 구조로청크 조각 → 목차 · 문맥 헤더 · 위키 링크 ·
지식 그래프

문서를 잘게 자르는 대신, 위치와 관계를 보존해 에이전트가 "어디를 볼지 정하고 따라가며 읽도록" 만듭니다.

LLM Wiki(Retrieval as Reasoning), 사람이 지식 영역을 정의하는 라우팅형 RAG

03 · 운영 방식

모든 질문에 같은 경로에서
난이도별 경로로단일 경로 → 쉬운 질문은 빠른 경로, 어려운
질문만 에이전트 루프

실제 질의의 60~70%는 단순 조회입니다. 에이전트 루프는 토큰 3~10배를 쓰므로, 필요한 곳에만 씁니다 (Adaptive RAG).

프로덕션 Agentic RAG 패턴: Router · ReAct · Plan-and-Execute · Multi-Agent

04 · 신뢰와 통제

"답이 나온다"에서
"증명하고 통제한다"로결과 채점 → 과정(궤적) 채점 · 권한 반영
검색 · 표준 연결(MCP)

도구 선택과 호출 횟수까지 평가하고, 사용자별 접근 권한을 검색에 반영하며, 사내 시스템은 MCP 같은 표준으로 연결합니다.

궤적 평가(LangSmith 등), Permission-aware RAG, MCP 인가

개발자의 핵심 역량은 "파이프라인을 정교하게 조립하는 능력"에서 "도구와 데이터를 설계하고, 평가로 개선을 증명하는 능력"으로 옮겨 가고 있습니다.

KEY ELEMENTS

Agentic RAG 구현을 위해 알아야 할 것

요소	핵심	스스로 던져야 할 설계 질문
에이전트 루프	모델 + 도구 + 시스템 프롬프트 + 대화 상태. 프레임워크로 몇 줄이면 구성됩니다 (예: <code>create_agent</code>)	루프에 무엇을 연결하는가?
도구 인터페이스	이름 · 설명 · 인자 · 반환 형식. 모델은 함수 코드가 아니라 이것만 봅니다	설명만 읽고 언제 쓸지 알 수 있는가?
데이터 계층	정규화 · 문맥 헤더 · 메타데이터. 데이터에 필드가 있어야 도구 인자가 의미를 가집니다	도구가 필요한 필터를 데이터가 갖고 있는가?
연결 표준	MCP로 데이터 소스를 서버로 분리하고, 인증과 권한을 겁니다	소스가 늘어도 에이전트 코드는 그대로인가?
평가	결과와 과정(도구 선택 · 호출 횟수 · 시간)을 함께 채점하고, 개선을 숫자로 증명합니다	정답뿐 아니라 과정과 비용도 나아졌는가?

에이전트를 만드는 코드는 짧습니다. **답변의 품질은 에이전트에게 어떤 도구를 주고, 그 도구가 어떤 데이터를 찾느냐에 달려 있습니다.**

DAY 1 · MODULE 2

엔터프라이즈 데이터 인덱싱

문서를 검색할 수 있는 데이터로 만들기: 정규화 · 청킹 · 하이브리드 검색 · 지식 그래프

MODULE OVERVIEW

2교시 학습 목표

• 인덱싱의 두 가지 문제와 정규화

- 청크가 맥락을 잃는 문제와 문서 형식이 제각각인 문제를, 표준 Markdown으로 먼저 맞춰서 해결합니다

• 청크가 스스로 출처를 말하게 하기

- 문맥 헤더(목차 · 현재 위치)와 메타데이터를 청크에 붙이는 방법을 다룹니다

• 의미와 키워드를 함께 찾는 검색

- Dense + BM25 하이브리드 검색, RRF 융합, 윈도우 확장으로 약어 · 조항 번호 · 표를 놓치지 않게 합니다

• 관계를 따라가는 검색: 지식 그래프와 GraphRAG

- 여러 문서에 흩어진 사실을 이어야 답할 수 있는 질문을 그래프로 풀니다

INDEXING PIPELINE

인덱싱 파이프라인 전체 그림

STEP 1

정규화

형식이 다른 문서를 같은 규격의 Markdown으로 바꿉니다.

PDF · DOCX · MD → 표준
Markdown



STEP 2

청킹

논리 단위로 자르고, 각 청크에 문맥
헤더와 메타데이터를 붙입니다.

목차 · 현재 위치 · 기간 태그



STEP 3

인덱스

같은 청크를 의미 검색용과 키워드
검색용으로 함께 저장합니다. 관계
정보는 그래프로 만듭니다.

Vector · BM25 · Graph



STEP 4

검색

질문 용어를 정리하고, 두 검색 결과를
합친 뒤, 앞뒤 문맥까지 붙여
돌려줍니다.

약어 사전 → RRF → 윈도우 확장

RAG 품질 문제의 상당수는 모델이 아니라 **문서 파싱과 인덱싱**에서 생깁니다. 에이전트는 잘못 잘린 청크나 끊어진 관계를 보완해 주지 못합니다.

TWO PROBLEMS

인덱싱의 두 가지 문제

PROBLEM 1

탈맥락화 (Decontextualization)

문서를 청크로 자르는 순간, 각 청크는 자신이 **어느 문서의 어느 위치**였는지 잃어버립니다.

"환불은 구매일로부터 7일 이내에 가능하다"
→ 어느 상품의, 어느 약관의 몇 조인가?

분기 보고서처럼 같은 문장 구조가 반복되는 문서는 더 심각합니다. "매출이 전년 대비 12% 증가했다"만으로는 어느 사업부, 어느 분기인지 알 수 없습니다.

비유: 책에서 찢어 낸 한 페이지. 책 제목도, 몇 장인지도 모릅니다

PROBLEM 2

포맷 파편화

규정은 Markdown, 보고서는 다단 편집 PDF, 계약서는 DOCX · HWP입니다. 형식마다 파싱 · 청킹 · 검색 코드를 따로 만들면 **문서 종류만큼 파이프라인이 늘어납니다.**

.md → 청킹기 A → 검색기 A
.pdf → 청킹기 B → 검색기 B
.docx → 청킹기 C → 검색기 C

형식마다 품질 기준도 달라져서, 어떤 문서가 잘 검색되고 어떤 문서가 안 되는지 관리하기 어려워집니다.

NORMALIZE FIRST

해법: 모든 문서를 먼저 같은 규격으로

.md 규정 · 매뉴얼

이미 계층이 있음

.pdf 보고서

계층 정보가 없음

.docx · .hwp 문서

스타일에 계층이 숨어 있음

**형식별 정규화기**형식마다 얇게 하나씩.
새 형식이 오면 정규화기만 추가**표준 Markdown 규격**

H1 문서 안의 최상위 주제

H2~H4 장 · 절 · 조항

본문 표는 Markdown 표로

metadata 계층에 없는 속성 (출처, 기간)

이후 청킹 · 인덱스 · 검색은 하나를 공용으로 사용

문제	정규화가 해결하는 방식
포맷 파편화	형식이 무엇이든 같은 규격으로 바꾸므로, 뒤쪽 파이프라인은 문서가 원래 무엇이었는데지 몰라도 됩니다
탈맥락화	규격에 계층(문서 > 장 > 조항)이 생기므로, 그 계층을 모든 청크에 문맥 헤더로 붙일 수 있습니다

PDF NORMALIZATION

PDF 정규화: 파서는 조용히 틀린다

일반 변환기의 한계

PDF에는 "이 글자가 제목"이라는 정보가 없고 글자 크기와 위치뿐입니다. 변환기는 제목을 추측하고, 그 과정에서 계층을 잘못 만들거나 텍스트 일부를 빠뜨립니다.

- 장식 문구가 제목으로 잡힘
- 진짜 섹션 제목은 본문으로 묻힘
- 에러도 경고도 없음

문서가 이미 가진 구조 정보를 쓴다

문서가 가진 정보	규격에서의 위치
PDF 전자 북마크(목차)와 시작 페이지	H1 (헤더 승격)
섹션마다 반복되는 소제목	H2
파일명 속 연도 · 분기	메타데이터
매 페이지 상단의 러닝 헤더	삭제 (노이즈)

검수는 눈이 아니라 숫자로

텍스트 보존율

원문 텍스트가 변환 결과에 빠짐없이 남았는가. 기준 예: 99% 이상

헤더 분포

규격에 없는 헤더(표지 문구, 각주 등)가 생기지 않았는가. 기준 예: 0개

PARSING IN PRACTICE

정규화의 한계와 역할 분담

남는 한계는 알고 쓴다

한계	예	대안
차트	그래프 속 수치는 텍스트가 아님	VLM 캡션, 원천 통계를 별도 적재
표	셀이 흩어지거나 값이 빠짐	레이아웃 분석 모델 (Docling, Upstage 등)
스캔 문서	기델 구조 정보가 없음	OCR · VLM 기반 파싱

어떤 대안을 쓰든 **바뀌는 것은 정규화기 하나**입니다. 출력이 같은 규격이면 이후 파이프라인은 그대로입니다.

문서 계열마다 파서를 만드는 비용은?

사람 (한 번)

목표 규격(무엇을 H1 · H2로 삼을지, 어떤 메타데이터를 붙일지)과 검수 기준을 정합니다.

AI 코딩 에이전트 (문서 계열마다)

문서 구조를 진단하고 정규화 설정을 작성한 뒤, 검수 기준을 통과할 때까지 고칩니다.

공용 파이프라인 (수정 없음)

같은 양식으로 계속 나오는 문서는 설정 하나로 계속 처리됩니다.

CONTEXTUAL CHUNKING

문맥 헤더: 청크가 스스로 출처를 말하게

임베딩 · 검색 대상이 되는 청크

[문서명: 00물 이용약관]

[소관부서: 고객지원팀]

[문서 전체 목차]: 제1장 총칙 · 제2장 주문 · 제3장 취소와 환불 · ...

[현재 위치: 00물 이용약관 > 제3장 취소와 환불 > 제12조 환불 기간]

2. 단순 변심에 의한 환불은 구매일로부터 7일 이내에 가능하다.

metadata.raw_content = 원본 청크만 따로 보관

두 단계로 자른다

① H1~H4 헤더 기준으로 논리 단위로 나누고, ② 긴 섹션만 글자 수 기준으로 다시 나눕니다.

현재 위치 (Breadcrumb)

청크가 어느 조항인지 스스로 설명합니다. "약관의 환불 기간" 같은 질문과 유사도가 올라갑니다.

문서 목차 (TOC)

문서 전체 구조를 함께 담아 어떤 문서의 일부인지 구분합니다. 다만 같은 문서의 청크끼리 비슷해지므로, 넣을지는 문서 특성에 따라 정합니다.

원본 보관 (raw_content)

헤더 없는 원문을 따로 두어, 답변 생성 때 헤더가 반복되지 않게 이어 붙입니다.

METADATA

문맥 헤더와 메타데이터 필터: 둘 다 붙인다

구분	문맥 헤더 (Soft)	메타데이터 필터 (Hard)
들어가는 곳	임베딩되는 텍스트 안	벡터 DB의 메타데이터 필드 (예: <code>quarter: 3</code>)
동작	유사도를 높여 관련 청크가 위로 오게 함	조건에 맞지 않는 청크를 아예 제외
강점	질문에 조건이 없어도 맥락으로 찾아감	다른 기간 · 다른 문서가 섞일 일이 없음
약점	본문이 비슷하면 다른 기간의 청크가 섞임	조건이 빠지거나 여러 기간을 비교할 때 과하게 막음

메타데이터는 인덱싱 때 붙여 두기만 하면 비용이 거의 없습니다. **필터를 언제 쓸지는 에이전트가 정하게** 검색 도구의 인자로 열어 둡니다. 예: `search_reports(query, quarter=None)`

HYBRID SEARCH

왜 벡터 검색만으로는 부족한가

질문	Dense (의미)	BM25 (키워드)	이유
"출장 가면 밥값은 얼마 줘?"	✓	✗	문서에는 "식비". 뜻은 같고 단어가 다름
"제15조 내용 알려줘"	△	✓	조항 번호는 뜻이 아니라 같은 글자 가 중요
"VPN 신청 절차"	△	✓	약어 · 고유명사는 임베딩에서 의미가 흐릿함
"CS팀 출장 결재"	△	△	문서에는 "고객지원팀". 약어 사전 으로 먼저 바꿔야 둘 다 찾을 수 있음

Dense는 **뜻이 같은 다른 표현**에 강하고, BM25는 **정확히 같은 글자**에 강합니다. 서로의 약점을 매우도록 둘을 함께 씁니다.

HYBRID SEARCH

하이브리드 검색 파이프라인

STEP 1 · 질의 정리

약어 사전

사내 약어 · 은어를 정식 명칭으로 바꿉니다. 규칙 기반이라 빠르고 비용이 없습니다.

CS팀 → 고객지원팀



STEP 2 · 병렬 검색

BM25 + Dense

같은 질문으로 키워드 검색과 의미 검색을 각각 실행합니다.

두 개의 순위 목록



STEP 3 · 융합

RRF

점수가 아니라 순위만으로 두 목록을 하나로 합칩니다.

선택: Rerank 모델로 상위 후보 재정렬



STEP 4 · 문맥 복원

윈도우 확장

찾은 청크의 앞뒤 청크를 붙여 잘린 표와 단서 조항을 복원합니다.

작게 찾고 크게 읽기

한국어 BM25 주의

공백 기준 토큰화는 조사("출장비는", "출장비를") 때문에 키워드가 어긋납니다. 형태소 분석기(예: Kiwi)로 명사 단위로 나눠 색인합니다.

HyDE · Rerank는 선택

가상 답변으로 검색하는 HyDE, 교차 인코더로 다시 정렬하는 Rerank는 정확도를 올리지만 호출 비용과 지연이 늘어납니다.

RECIPROCAL RANK FUSION

RRF: 왜 점수가 아니라 순위로 합치나

점수는 단위가 다르다

Dense 점수는 코사인 유사도(0~1), BM25 점수는 0~30 이상입니다. 그대로 더하면 BM25가 결과를 좌우합니다. RRF는 점수를 버리고 각 목록에서의 순위만 씁니다.

$$\text{RRF}(d) = \sum \frac{1}{k + \text{rank}(d)} \quad (\text{보통 } k = 60)$$

청크	BM25 순위	Dense 순위	RRF 점수	최종
A	1	2	$1/61 + 1/62 = 0.0325$	1위
C	3	3	$1/63 + 1/63 = 0.0317$	2위
B	—	1	$1/61 = 0.0164$	3위

한쪽에서만 1위인 청크(B)보다, **양쪽에서 고르게 상위**인 청크(A, C)가 앞에 옵니다. 두 검색기가 서로의 실수를 걸러 줍니다.

WINDOW EXPANSION

작게 찾고 크게 읽기: 윈도우 확장

청크 크기	검색 정확도	답변 품질
작게 (300~500자)	주제가 하나라 유사도가 선명함	표가 중간에 잘리고, 단서 조항이 다음 청크로 넘어감
크게 (2,000자 이상)	여러 주제가 섞여 유사도가 흐려짐	맥락이 온전함

1단계

작은 청크로 검색

청크 15가 검색에 걸림



2단계

앞뒤 청크 조회

인덱싱 때 만든 (문서, 청크 번호) 조회표로 14 · 16번을 바로 가져옴



3단계

하나로 결합

문맥 헤더 1회 + 14 · 15 · 16번 원문을 이어 붙여 모델에 전달

비유하면, 사전에서 단어를 찾은 뒤 **앞뒤 문단까지 함께 읽는 것**입니다. 앞뒤로 몇 개를 붙일지(window size)는 맥락과 토큰 비용 사이의 선택입니다.

KNOWLEDGE GRAPH

흩어진 사실을 이어야 답할 수 있는 질문

질문: "프로젝트 X의 PM이 속한 본부의 본부장은 누구인가?"

문서 곳곳에 흩어진 사실

"홍길동 팀장은 프로젝트 X의 PM을 맡고 있다." (프로젝트 현황)

"홍길동 팀장은 데이터플랫폼팀 소속이다." (조직도)

"데이터플랫폼팀은 AI본부 산하이며, 본부장은 이영희 전무다." (조직 개편 공지)

어떤 청크도 "프로젝트 X → 이영희"를 직접 담고 있지 않아, 벡터 검색은 중간 연결을 놓칩니다.

(주어, 관계, 목적어) 트리플렛으로 추출

[홍길동] -(PM)→ [프로젝트 X]
 [홍길동] -(소속팀)→ [데이터플랫폼팀]
 [데이터플랫폼팀] -(소속본부)→ [AI본부]
 [AI본부] -(본부장)→ [이영희]

같은 이름의 노드가 다리가 되어, 서로 다른 문서의 사실이 하나의 연결망으로 이어집니다.

그래프를 따라가면 답이 나온다

프로젝트 X

↓ PM

홍길동

↓ 소속팀

데이터플랫폼팀

↓ 소속본부

AI본부

↓ 본부장

★ 이영희

BUILDING A KNOWLEDGE GRAPH

지식 그래프를 만들 때 정할 것

ONTOLOGY

관계 어휘를 먼저 정한다

제약 없이 뽑게 하면 "이끈다", "팀장이다",
"수장이다"처럼 같은 관계가 여러 단어로 갈라져
그래프 검색이 안 됩니다.

엔티티 종류(인물 · 부서 · 프로젝트)와 관계(소속팀
· 소속본부 · PM · 승인권자)를 스키마로 고정해
추출

ENTITY RESOLUTION

같은 대상은 한 노드로

"홍길동 팀장", "홍길동 수석", "홍 팀장"이 서로 다른
노드가 되면 연결이 끊깁니다.

추출 후 이름 표기를 정규화해 하나의 노드로 합침.
실무에서는 별도 단계로 둬

HOP DEPTH

탐색 깊이는 질문마다 다르다

1-Hop은 직접 연결만 봅니다. 2-Hop 이상이면 먼
관계까지 닿지만 방문 노드가 빠르게 늘어
컨텍스트가 지저분해집니다.

깊이를 검색 도구의 인자(1~3)로 열고, 에이전트가
질문에 맞게 고르게 함

트리플렛 추출은 LLM이 하지만, **무엇을 노드로 삼고 어떤 관계만 허용할지는 사람이 정합니다.** 이 설계가 그래프 검색의 품질을 좌우합니다.

KG · GRAPHRAG

Knowledge Graph와 GraphRAG는 어떻게 다른가

용어	정체	설명
Knowledge Graph	데이터 구조	엔티티(노드)와 관계(엣지)로 지식을 저장한 것. 트리플렛의 집합
GraphRAG	검색 방법의 총칭	그래프를 활용해 검색하고 답을 만드는 RAG. 대부분 내부에 Knowledge Graph를 만듭니다
Microsoft GraphRAG	대표 구현체	LLM으로 KG를 만든 뒤 커뮤니티를 묶고, 커뮤니티마다 요약 리포트 까지 미리 생성

같은 그래프, 다른 질의 방식

지식 그래프 직접 질의

- 시작 엔티티에서 N홉 엣지를 코드로 따라감
- 검색할 때 LLM 호출 없음 → 빠르고 결과가 일정
- "A의 상위 조직은?"처럼 **정확한 관계**에 적합

GraphRAG 질의 (Microsoft GraphRAG)

- 엔티티 주변 맥락이나 커뮤니티 요약에 LLM이 종합
- 검색할 때도 LLM 호출 → 느리고 비용이 큼
- "전사 프로젝트의 공통 리스크는?"처럼 **넓은 요약**에 적합

MICROSOFT GRAPHRAG

Microsoft GraphRAG의 인덱싱 과정

STEP 1

엔티티 · 관계 추출

청크마다 LLM이 트리플렛과 함께 엔티티 · 관계의 **설명문**까지 뽑습니다.

홍길동 (인물): "데이터플랫폼팀 팀장, 프로젝트 X PM"



STEP 2

그래프 병합

여러 청크에서 나온 같은 엔티티 · 관계를 하나로 합치고, 설명문을 모아 요약합니다.

문서 전체에 걸친 지식이 한 노드에 쌓임



STEP 3

커뮤니티 묶기

Leiden 알고리즘으로 서로 밀접한 노드를 계층적인 그룹(커뮤니티)으로 나눕니다.

전사 → 본부 → 팀 · 프로젝트 단위



STEP 4

커뮤니티 리포트

커뮤니티마다 LLM이 제목 · 요약 · 주요 발견을 담은 리포트를 미리 써 둡니다.

Global 검색의 재료

비용 주의

추출 · 요약 단계마다 LLM을 호출하므로 인덱싱 비용이 큼니다. 자주 바뀌지 않는 문서 묶음에 적합합니다.

대안: 질의 시점에 그래프 만들기

LazyGraphRAG처럼 인덱싱 때 요약을 미리 만들지 않고, 질문이 들어올 때 필요한 부분만 처리하는 방식도 나왔습니다.

LOCAL VS GLOBAL

GraphRAG의 두 가지 검색: Local과 Global

LOCAL SEARCH

특정 엔티티에 대한 질문

"홍길동 팀장의 역할과 보고 라인은?"

- 질문 속 엔티티와 비슷한 노드를 찾음
- 주변 관계(트리플렛 + 설명), 원문 청크, 소속 커뮤니티 요약물 함께 모음
- 모은 맥락으로 LLM이 답변

GLOBAL SEARCH

시작점이 없는 넓은 질문

"전사 프로젝트들의 공통 리스크는?"

- 특정 노드에서 출발할 수 없으므로 커뮤니티 리포트 전체를 사용
- Map: 리포트별로 관련 내용을 뽑음
- Reduce: 뽑은 내용을 하나의 답으로 종합

직접 만든 지식 그래프는 "누가 누구와 어떻게 연결되었나"에, Global Search는 **"전체를 훑어야 답할 수 있는 질문"**에 강합니다. 질문 유형에 따라 나눠 씁니다.

KEY ELEMENTS

2교시 정리: 데이터와 질문에 맞는 인덱스

데이터 · 질문	인덱스	핵심 기법	검색 도구로 열 인자
규정 · 매뉴얼 본문	Vector + BM25 하이브리드	문맥 헤더, 약어 사전, RRF	<code>query</code>
기간별 반복 보고서	Vector + 메타데이터	정규화, 기간 태그	<code>query</code> , <code>quarter</code>
사람 · 조직 관계	지식 그래프 (직접 질의)	관계 어휘 고정, 엔티티 정규화	<code>entity</code> , <code>depth</code>
전체 요약 질문	Microsoft GraphRAG	커뮤니티 리포트	<code>question</code> , <code>mode</code>

어느 인덱스가 가장 좋은지 고르는 문제가 아닙니다. **데이터마다 맞는 인덱스를 만들고, 각각을 검색 도구로 노출**하면 에이전트가 질문에 따라 골라 씁니다.

DAY 1 · MODULE 3

MCP & Skills: 다중 데이터 소스 연동

여러 데이터 소스를 도구로 연결하고, 서버로 분리하고, 필요할 때 찾아 쓰게 하기

MODULE OVERVIEW

3교시 학습 목표

• 여러 데이터 소스를 도구로 연결하기

- 문서 · 관계 · 정형 데이터 · 업무 시스템을 각각 어떤 검색 도구로 만들지 설계합니다

• MCP로 데이터 접근 계층 분리하기

- 도구를 에이전트 코드에서 떼어 표준 서버로 만들고, 여러 에이전트가 함께 쓰게 합니다

• 인증과 인가로 접근 통제하기

- 누가 서버에 들어올 수 있고, 들어온 뒤 어떤 도구를 쓸 수 있는지 나눠서 관리합니다

• 도구가 많아질 때: 정적 바인딩 vs Skills

- 모든 도구를 미리 주는 방식과, 필요할 때 설명서를 읽고 찾아 쓰는 방식을 비교합니다

MULTI-SOURCE

여러 소스를 연결하는 두 가지 방식

방식 A

규칙 기반 라우터

질문 → 의도 분류기 → 정해진 핸들러 하나 실행

- + 빠르고 결과를 예측하기 쉽습니다
- + 질문 유형이 적고 고정적일 때 적합합니다
- 한 질문에 소스 하나만 고릅니다
- 질문 유형이 늘면 분류 규칙도 계속 늘어납니다

방식 B

에이전트의 도구 선택

질문 → 모델이 도구 설명을 읽고 선택 → 결과를 보고 다음 도구

- + 한 질문에 여러 소스를 순서대로 조합할 수 있습니다
- + 새 소스는 도구를 추가하는 것으로 연결됩니다
- 호출이 늘어나는 만큼 비용과 시간이 듭니다
- 도구 설명이 부실하면 엉뚱한 도구를 고릅니다

실무에서는 둘을 섞어 씁니다. **단순하고 자주 오는 질문은 라우터로 빠르게, 여러 소스가 필요한 질문은 에이전트로** 처리합니다.

TOOL PER SOURCE

소스 유형별 검색 도구 설계

소스	잘 맞는 질문	도구 인자 예	설계 포인트
문서 (Vector + 키워드)	"규정에 뭐라고 되어 있나?"	query, 기간 · 문서 종류 필터	출처를 함께 반환. 필터는 인덱스의 메타데이터와 짝을 맞춤
관계 탐색 (지식그래프 직접 질의)	"A의 상위 조직은?" "B와 연결된 프로젝트는?"	시작 엔티티, 탐색 깊이	탐색 깊이에 상한을 둬. 엔티티 이름 표기를 통일
전체 요약 (GraphRAG Global)	"전사 프로젝트의 공통 리스크는?"	질문, 범위 (Local / Global)	비용이 크므로 전체를 훑어야 하는 질문에만 사용
정형 데이터 (SQL)	"부서별 지난달 집행액은?"	자연어 질문 또는 SQL	읽기 전용, 반환 행 수 제한, 관련 테이블만 노출
업무 시스템 (API)	"이 요청 건의 처리 상태는?"	문서 번호, 상태	조회와 변경을 분리. 변경 작업은 사람의 승인을 거침

관계 탐색과 GraphRAG는 **모두 Knowledge Graph를 씁니다**. 다른 것은 질의 방식입니다. 직접 질의는 엣지를 코드로 따라가고(LLM 없음), GraphRAG는 엔티티 주변 맥락이나 커뮤니티 요약을 LLM으로 종합합니다.

TEXT2SQL

정형 데이터를 도구로: Text2SQL의 세 가지 보완

STEP 1

질문 명확화

"지난달 실적 좋은 부서"처럼 기준이 빠진 질문에서
기간 · 지표 · 기준일을 먼저 확정합니다. 모호하면
되물습니다.

"좋은 실적" = 어떤 지표, 어떤 기간?



STEP 2

관련 테이블 선별

수백 개 테이블을 모두 보여 주지 않고, 질문과 관련된
2~3개 테이블과 연결 관계만 모델에게 줍니다.

스키마를 줄일수록 잘못된 조인이 줄어듦



STEP 3

업무 용어 사전 (Semantic Layer)

회사에서 쓰는 지표 정의와 계산식을 컬럼과 조건으로
번역해 둡니다. 모델이 계산식을 추측하지 않게 합니다.

가용 예산 = 배정 예산 - 집행 금액

안전장치: 프롬프트로 부탁하지 말고 도구 안에서 강제

읽기 전용 계정

DB 계정 권한 자체를 조회로
제한합니다.

SELECT만 허용

INSERT · UPDATE · DROP이 섞이면
실행하지 않습니다.

반환 행 수 제한

LIMIT를 강제해 결과가 컨텍스트를
넘치지 않게 합니다.

실행 전 검증

문법 · 테이블 이름을 먼저 확인하고,
오류는 이유와 함께 돌려줍니다.

TOOL DESIGN

좋은 검색 도구의 다섯 가지 조건

조건	나쁜 예	좋은 예
이름과 설명이 구체적	<code>search(query)</code> "검색합니다"	<code>search_policy_docs</code> "규정 조항을 검색. 금액 · 조건을 확인할 때 사용"
인자 범위를 제한	<code>depth: int</code> (아무 값이나 허용)	<code>depth: 1~3</code> (스키마로 강제, 벗어나면 거부)
결과가 없으면 알려줌	빈 문자열 반환	"'OO' 관련 문서를 찾지 못했습니다" → 모델이 다음 행동을 정함
출처를 함께 반환	본문만 반환	[문서명 > 조항] + 본문 → 답변에 근거 인용 가능
필요한 만큼만 반환	문서 전체를 통째로 반환	상위 몇 건, 핵심 필드만 → 컨텍스트 절약

도구 설명은 **처음 온 동료에게 건네는 업무 메모**와 같습니다. 모델은 함수 코드를 보지 않고, 이름 · 설명 · 인자만 보고 언제 쓸지 정합니다.

WHY MCP

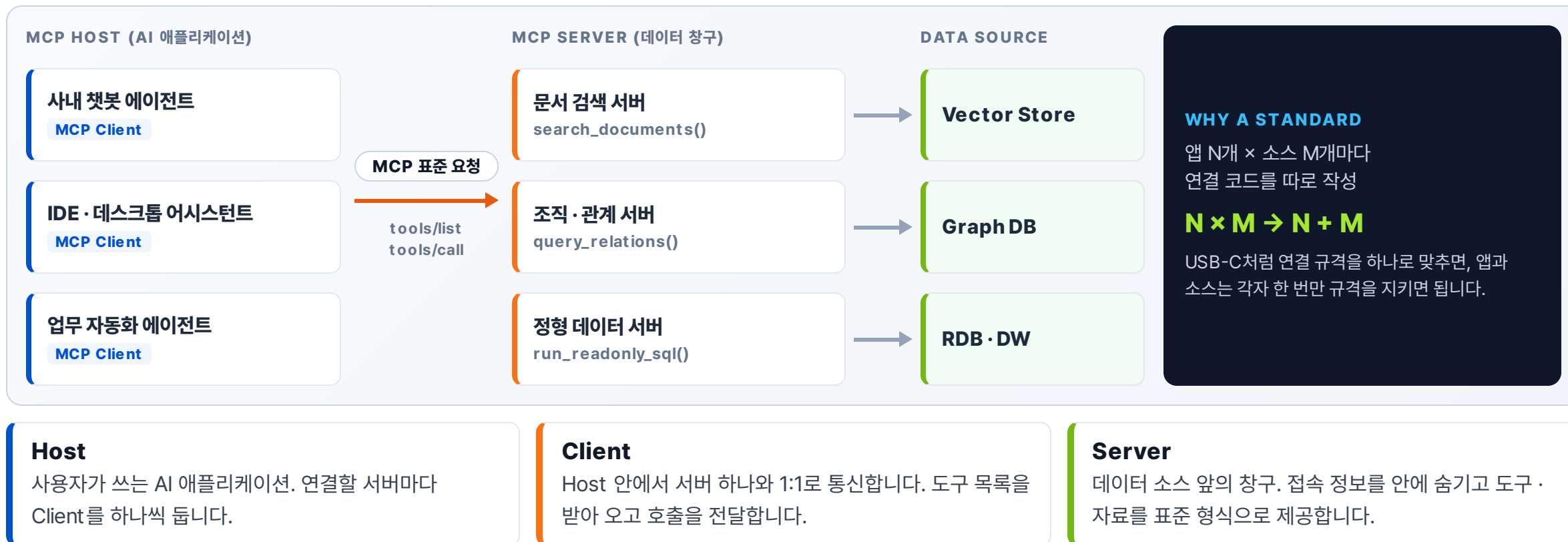
도구가 에이전트 코드 안에 있으면 생기는 문제

관점	도구를 에이전트 코드에 직접 포함	도구를 MCP 서버로 분리
소유와 배포	검색 로직을 고칠 때마다 에이전트 전체를 다시 배포	데이터 담당 팀이 서버만 따로 배포, 에이전트는 그대로
권한과 보안	에이전트가 모든 DB 접속 정보를 직접 보유	서버가 접속 정보를 갖고 인증 · 인가의 관문 역할
재사용	에이전트마다 같은 검색 코드를 복사	챗봇 · IDE · 업무 자동화가 같은 서버를 공유
기술 선택	에이전트와 같은 언어 · 런타임으로 제한	데이터 소스에 맞는 언어로 서버 구현 가능

비유하면, 모든 부서 업무를 한 사람이 직접 처리하던 구조에서 **부서별 창구를 두고 정해진 양식으로 요청**하는 구조로 바꾸는 것입니다.

MCP ARCHITECTURE

MCP 구조: Host · Client · Server



MCP PRIMITIVES

MCP 서버가 제공하는 것과 연결 방식

서버가 제공하는 세 가지

Tools (도구)

모델이 호출하는 함수. 검색 · 조회 · 계산처럼 실행이 필요한 일. 인자는 스키마로 정의하고 서버가 검증합니다.

예: `search_documents(query, period)`

Resources (자료)

읽기 전용으로 참조하는 데이터. 파일, 테이블 스키마, 문서처럼 그대로 읽어 오는 정보입니다.

예: 매출 테이블 스키마, 용어 사전

Prompts (프롬프트)

서버가 미리 만들어 배포하는 프롬프트 템플릿. 자주 쓰는 질의 형식을 표준화합니다.

예: "월간 실적 요약" 템플릿

연결 방식 (Transport)

stdio (로컬)

Host가 서버를 하위 프로세스로 띄우고 표준 입출력으로 통신합니다. 데스크톱 앱, CLI 도구, 개인 개발 환경에 적합합니다.

예: 개인 PC의 IDE가 파일 검색 서버를 직접 실행

Streamable HTTP (원격)

서버를 독립된 웹 서비스로 띄우고 HTTP로 통신합니다. 로드밸런서 · 게이트웨이 뒤에 둘 수 있어 사내 운영 환경의 기본 선택입니다.

예: 사내 클라우드에 배포한 서버에 여러 에이전트가 URL로 접속

ACCESS CONTROL

인증(Authentication)과 인가(Authorization)

AUTHENTICATION

인증: "누구인가?"

요청한 쪽이 유효한 자격 증명(토큰)을 가졌는지 확인합니다. 통과하지 못하면 서버에 들어올 수 없습니다.

비유: 건물 입구에서 직원증을 확인하는 것

AUTHORIZATION

인가: "이것을 해도 되는가?"

들어온 사람이라도 권한 범위(scope)에 따라 쓸 수 있는 도구와 자료를 제한합니다.

비유: 직원증마다 들어갈 수 있는 층과 방이 다른 것

요청이 도구 실행까지 가는 과정

1

요청

Client가 토큰을 헤더에 담아 보냄



2

인증

토큰이 유효한가?

실패 → 401 Unauthorized



3

인가

이 도구를 쓸 scope가 있는가?

실패 → 403 Forbidden



4

도구 실행

요청자 정보로 결과 필터링 · 감사 로그 기록

TOKEN VERIFICATION

토큰 검증 방식과 RemoteAuthProvider

① StaticTokenVerifier · 개발용

등록해 둔 토큰 목록과 비교합니다. 토큰이 코드에 그대로 남기 때문에 개발 · 테스트에서만 씁니다.

② JWTVerifier · 운영용 검증기

IdP가 공개한 주소(`jwt_uri`)에서 공개키를 받아 두고, 들어온 토큰의 서명 · 발급자 · 대상 · 만료를 확인합니다. 검증만 하기 때문에 에이전트가 토큰 받을 곳을 미리 알고 있어야 합니다.

③ RemoteAuthProvider · 검증기 + IdP 안내

토큰 검증은 ②에 맡기고, "토큰은 이 IdP에서 받으세요"라는 안내를 서버 주소로 공개합니다. IdP에 값을 보내는 설정이 아닙니다.

RemoteAuthProvider는 클라이언트 자동 등록(DCR)을 지원하는 IdP에 씁니다 (Keycloak, Okta 등). Azure AD · Google처럼 지원하지 않으면 **OAuthProxy**를 씁니다.

```
# ① 개발용: 등록해 둔 토큰과 비교
verifier = StaticTokenVerifier(tokens={
    "dev-token": {"client_id": "rag-agent",
                  "scopes": ["rag:read"]},
})

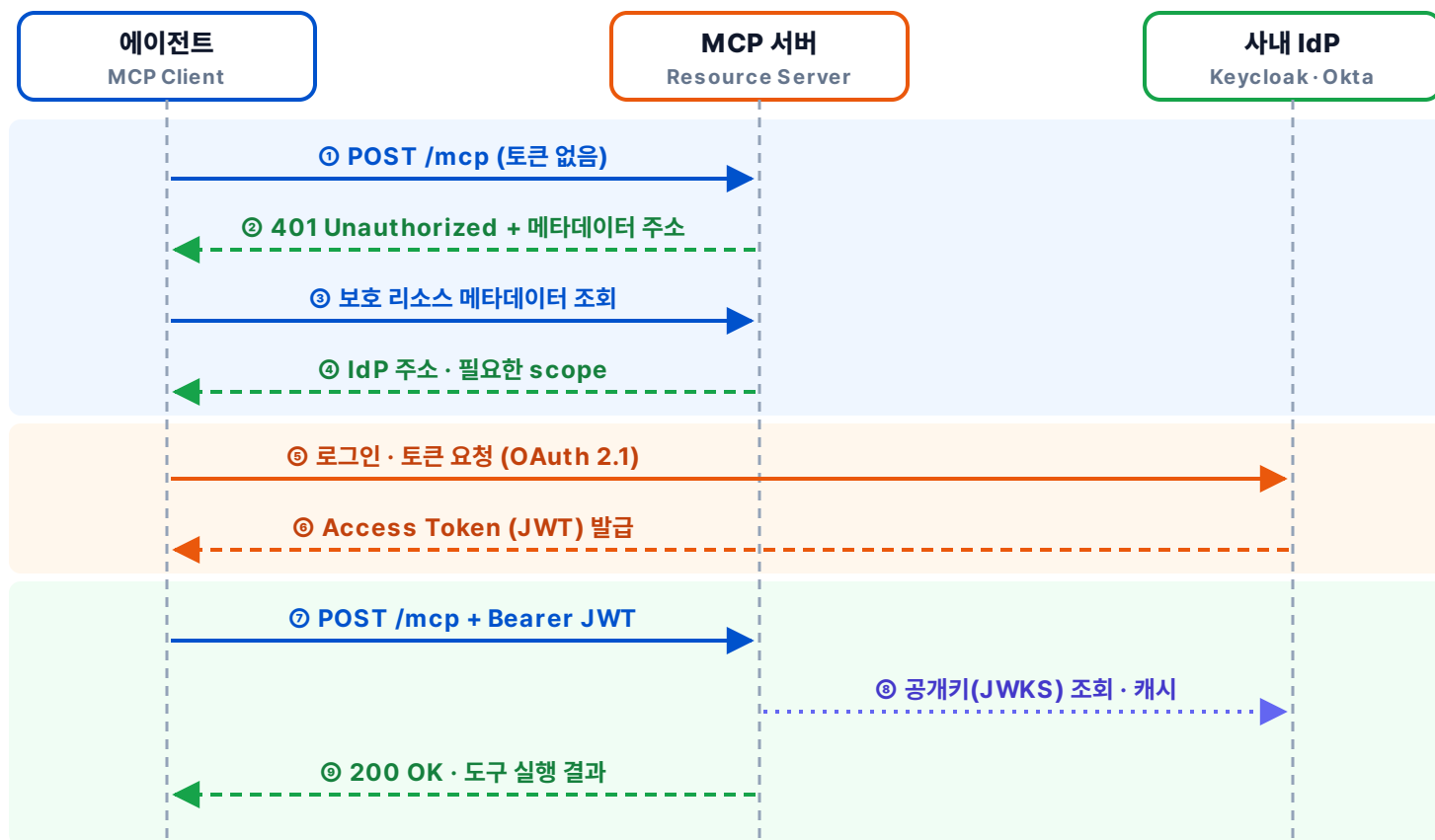
# ② 운영: IdP 공개키로 JWT 검증 (키는 jwt_uri에서 받아 캐시)
verifier = JWTVerifier(
    jwt_uri="https://idp.company.com/.well-known/jwks.json",
    issuer="https://idp.company.com",      # 발급자 확인
    audience="mcp-rag-server",            # 이 서버용 토큰인지
)

# ③ 검증은 verifier에 맡기고, IdP 주소를 메타데이터로 공개
# (IdP로 보내는 값이 아님)
auth = RemoteAuthProvider(
    token_verifier=verifier,
    authorization_servers=["https://idp.company.com"],
    base_url="https://mcp.company.com",
)
mcp = FastMCP("rag-server", auth=auth)

# 도구 쪽 코드는 검증 방식과 관계없이 동일
@mcp.tool(auth=require_scopes("rag:read"))
```

AUTH FLOW WITH IDP

에이전트 · MCP 서버 · IdP 사이에서 오가는 요청



발견 ①~④

토큰 없이 요청하면 서버가 401과 함께 메타데이터 위치를 알려 줍니다. 메타데이터에는 IdP 주소와 필요한 scope가 들어 있습니다.

```
WWW-Authenticate: Bearer resource_metadata=
"https://mcp.company.com/.well-known/
oauth-protected-resource"
```

토큰 발급 ⑤~⑥

사람이 쓰는 앱은 로그인 창(Authorization Code + PKCE), 서버 간 호출은 Client Credentials로 토큰을 받습니다. MCP 서버는 로그인에 관여하지 않습니다.

검증 · 호출 ⑦~⑨

서버는 서명 · 발급자 · 대상 · 만료 · scope를 확인하고 통과하면 도구를 실행합니다. 공개키는 캐시해 두고 매 요청마다 IdP를 부르지 않습니다.

AUTH IN PRODUCTION

운영 환경에서 함께 챙길 세 가지

SECRET

비밀값은 프롬프트에 넣지 않는다

에이전트에게는 토큰 값 대신 **토큰이 담긴 환경 변수 이름**만 알려 줍니다. 실제 값은 실행 스크립트나 Client가 읽어 헤더에 넣습니다.

막는 것: 대화 기록 · 로그 · 트레이스 유출,
"프롬프트를 그대로 출력해" 같은 인젝션

USER IDENTITY

사용자 신원을 서버까지 전달한다

"에이전트가 접근 가능한가"를 넘어 "**질문한 사람이 볼 수 있는가**"를 확인하려면, 토큰에 사용자 정보가 담겨 서버까지 전달되어야 합니다.

서버는 토큰의 사용자 · 부서 정보로 검색 결과를 걸러서 반환

LEAST PRIVILEGE

권한은 좁게, 토큰은 짧게

조회(`rag:read`)와 변경(`rag:write`) scope를 나누고, 에이전트에는 필요한 scope만 줍니다. 토큰 만료 시간은 짧게 둡니다.

토큰이 새어도 할 수 있는 일과 쓸 수 있는 시간을 줄임

MCP 서버는 데이터 앞의 **관문**입니다. 누가(인증), 무엇을(인가), 누구의 권한으로(사용자 신원) 요청했는지를 서버에서 확인해야 에이전트가 늘어나도 통제가 유지됩니다.

SCALING PROBLEM

서버가 늘어나면: 도구 폭발 (Tool Explosion)

컨텍스트 낭비

서버 10개 × 도구 10개면 100개 도구의 설명과 스키마가 매 요청마다 프롬프트에 들어갑니다. 질문 하나에 수만 토큰이 기본으로 쓰입니다.

예 질문은 한 줄인데, 함께 보내는 도구 설명이 수만 토큰

도구 선택 혼란

비슷한 이름과 설명의 도구가 많아지면 모델이 엉뚱한 도구를 고르거나 인자를 잘못 만드는 일이 늘어납니다.

예 `search_docs.find_documents` · `query_kb` 중 무엇을 써야 하나?

강한 결합

서버가 추가되거나 도구 스펙이 바뀔 때마다 에이전트의 연결 설정을 고치고 다시 배포해야 합니다.

예 인사 시스템 서버 하나 추가 → 이를 쓰는 모든 에이전트 재배포

비유하면, 출근 첫날 **업무 매뉴얼 100권을 책상에 쌓아 두고 시작하는 것**입니다. 게이트웨이로 서버를 하나로 묶으면 주소와 이름 충돌은 정리되지만, 모델이 읽어야 할 도구는 그대로입니다.

PROGRESSIVE DISCLOSURE

Skills: 필요한 순간에 필요한 만큼만 보여 주기

1단계 · 목록

스킬 이름과 한 줄 설명

어떤 스킬이 있는지만 항상 프롬프트에 둡니다.

항상 · 수십 토큰

2단계 · 설명서

SKILL.md 본문

→ 필요하다고 판단한 스킬의 설명서만 읽습니다. 사용 절차와 명령 예시가 들어 있습니다.

필요할 때 · 수백 토큰

3단계 · 실행

스크립트로 조회 · 실행

→ 설명서대로 스크립트를 실행해 서버의 도구 목록을 보고, 필요한 도구만 호출합니다.

실행할 때 · 결과만

비유하면, 도서관에서 **안내판(목록)**을 보고 → **필요한 책(설명서)**을 꺼내 → **해당 페이지(도구)**만 펼치는 것입니다. 에이전트가 처음부터 가진 것은 파일 찾기 · 파일 읽기 · 명령 실행 같은 범용 도구 몇 개뿐입니다.

모든 도구를 미리 줄 때

매 요청: 도구 100개의 설명 + 스키마 전체

Skills로 줄 때

매 요청: 스킬 목록 몇 줄 → 필요할 때만 설명서 1개 + 도구 1~2개

SKILLS IN ACTION

Skills의 구성과 실행 흐름

SKILLS STANDARD FOLDER TREE

```
skills/
├── mcp/ # MCP 서버 연동 스킬
│   ├── SKILL.md # [필수] 이름·설명 + 사용 절차
│   ├── scripts/ # [실행] CLI 스크립트
│   │   ├── list_tools.py # 도구 목록 조회
│   │   └── execute_tool.py # 도구 실행
│   ├── references/ # [참조] 필요할 때 읽는 지식
│   │   └── mcp_servers.json # 서버 레지스트리
│   └── assets/ # [자산] 템플릿 · 스키마
├── jupyter-notebook/ # 노트북 조작 스킬
│   └── SKILL.md
└── pdf/ # PDF 추출 · OCR 스킬
    └── SKILL.md

# mcp_servers.json
{"servers": {"rag-server": {
  "url": "https://mcp.company.com/mcp",
  "token_env": "RAG_MCP_TOKEN", ...}}}
```

1

스킬 찾기 **glob**

목록에서 "MCP 서버 연동" 스킬이 필요하다고 판단

2

설명서 읽기 **read SKILL.md**

스크립트 사용법과 인자 형식을 확인

3

서버 확인 **read references/mcp_servers.json**

서버 URL과 토큰이 담긴 환경 변수 이름을 확인

4

도구 목록 조회 **bash list_tools**

서버가 제공하는 도구와 인자 스키마를 받아 옴

5

도구 실행 **bash execute_tool**

필요한 도구만 호출하고 결과로 답변

스킬에 필요한 것은 **모두 스킬 폴더 안에** 있습니다. 새 서버는 `mcp_servers.json`에 항목을 추가하면 연결되고, 폴더째 다른 에이전트에 옮겨도 그대로 동작합니다.

TRADE-OFF

정적 바인딩 vs Skills: 언제 무엇을 쓰나

비교 항목	정적 바인딩 (모든 도구를 미리 연결)	Skills (필요할 때 찾아서 실행)
프롬프트 토큰	모든 도구 스키마가 매 요청마다 들어감	스킬 목록 몇 줄, 나머지는 필요할 때만
응답 속도	빠름 : 도구를 이미 알고 있어 바로 호출	느림: 설명서 읽기 → 목록 조회 → 실행 단계를 거침
도구 수 확장	수십 개를 넘으면 선택 정확도가 떨어짐	서버 · 도구가 많아도 목록으로 관리
새 서버 추가	에이전트 설정 수정 후 재배포	서버 레지스트리(mcp_servers.json)에 항목 추가
잘 맞는 곳	도구 몇 개로 끝나는 단일 목적 서비스, 응답 속도가 중요한 서비스	여러 부서 시스템을 넘나드는 사내 플랫폼, 도구가 자주 바뀌는 환경

둘 중 하나만 고를 필요는 없습니다. **자주 쓰는 핵심 도구는 직접 연결하고, 나머지는 Skills로 찾아 쓰게** 섞는 구성이 일반적입니다.

KEY ELEMENTS

3교시 정리: 연결 계층을 설계할 때 던질 질문

주제	핵심	설계 질문
다중 소스 연결	소스마다 성격에 맞는 검색 도구를 만들고, 모델이 고르게 합니다	이 질문에 필요한 소스를 모델이 구분할 수 있는가?
도구 설계	구체적인 설명, 제한된 인자, 출처 포함, 실패 메시지	설명만 읽고 언제 쓸지 알 수 있는가?
MCP 분리	데이터 접근을 표준 서버로 떼어 독립 배포 · 공유	소스가 바뀌어도 에이전트를 다시 배포하지 않아도 되는가?
접근 통제	인증으로 들어오는 사람을, 인가로 쓸 수 있는 도구를 제한	질문한 사람이 볼 수 없는 자료가 답변에 섞이지 않는가?
확장 방식	핵심 도구는 직접 연결, 나머지는 Skills로 점진적 공개	도구가 100개가 되어도 프롬프트가 감당되는가?

DAY 1 · MODULE 4

평가 기반 자가 개선 (Self-Correction)

에이전트를 채점하고, 원인을 찾아 고치고, 다시 채점해 개선을 증명하기

MODULE OVERVIEW

4교시 학습 목표

• 두 가지 Self-Correction 구분하기

- 실행 중에 답변 하나를 고치는 것과, 평가 결과로 시스템 자체를 고치는 것의 차이를 이해합니다

• 골든셋과 2계층 채점으로 에이전트 평가하기

- 최종 답만이 아니라 도구 선택 · 검색어 · 호출 횟수까지 함께 채점합니다

• 궤적으로 원인 계층 찾고, 고치고, 회귀 확인하기

- 문제가 프롬프트 · 인덱스 · 도구 중 어디에 있는지 찾고, 고친 뒤 전체를 다시 채점합니다

• 운영 환경의 개입 장치 설계하기

- 비싸거나 위험한 도구 앞의 사람 승인(HITL), CI 품질 게이트, 운영 트래픽 샘플 채점

TWO KINDS

두 가지 Self-Correction

구분	실행 중 자가 수정	평가 기반 개선
누가 고치나	에이전트가 실행 중에 스스로	개발팀이 평가 결과를 보고
무엇을 고치나	이번 답변 하나 (재검색, 답변 재작성)	시스템 자체 (도구, 인덱스, 프롬프트)
비용	매 요청마다 지연 시간과 LLM 호출이 늘어남	오프라인에서 한 번
효과의 지속	다음 요청에서 같은 실패가 반복될 수 있음	고친 내용이 모든 요청에 적용

"결과가 부족하면 다시 찾기"는 요즘 모델이 ReAct 루프 안에서 대부분 알아서 합니다. 더 중요한 것은 **에이전트가 스스로 고칠 수 없는 결함**, 즉 도구와 데이터 계층의 문제를 찾아내는 장치입니다.

RUNTIME CORRECTION

실행 중 자가 수정을 돕는 세 가지 층

PROMPT

지침으로 유도

"근거가 없으면 '확인되지 않는다'고 답하라", "결과가 부족하면 검색어를 바꿔라"처럼 판단 기준을 알려 줍니다.

쉽게 적용 · 지키는지는 모델에 달림

TOOL OUTPUT

판단 근거를 결과에 담기

빈 결과 대신 "찾지 못했다"는 메시지, 출처와 위치, 건수를 돌려줘 모델이 다음 행동을 정하기 쉽게 합니다.

도구 코드만 고치면 됨 · 효과가 큼

MIDDLEWARE

코드로 강제

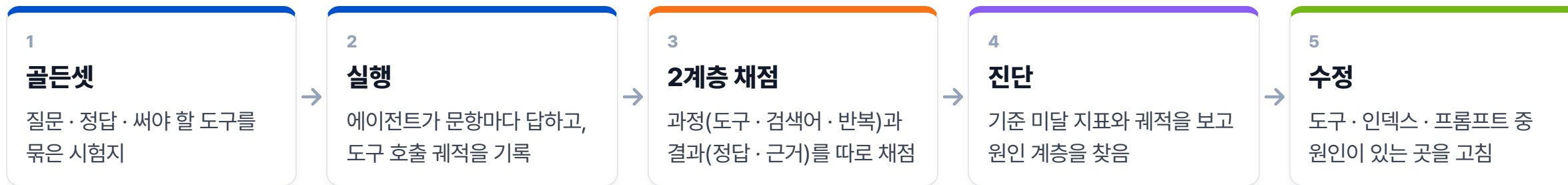
재시도 상한, 도구 호출 예산, 답변 전 근거 검사처럼 모델의 판단과 관계없이 지켜야 할 규칙을 실행 흐름에 끼웁니다.

확실함 · 매 요청 지연과 비용 증가

미들웨어 가드레일은 모든 요청을 느리게 만듭니다. 평가로 **그 실패가 실제로 자주 일어난다는 것을 확인한 뒤**, 금융 · 법무 · 인사처럼 틀린 답의 대가가 큰 곳에만 컵니다.

EVALUATION HARNESS

평가 하네스: 시스템을 고치는 루프



↻ 재평가: 고친 문항이 좋아졌는가(개선) + 다른 문항이 나빠지지 않았는가(회귀)

비유하면 모의고사입니다. 점수만 보고 끝내지 않고, 틀린 문제의 풀이 과정을 보고 약한 단원을 보강한 뒤 다시 시험을 봅니다.

GOLDEN DATASET

골든셋: 실패할 수 있는 방식마다 문항을 둔다

문항마다 붙이는 정답 라벨

정답 (ground_truth)

답변의 핵심 사실이 맞는가(정답 일치), 필요한 근거를 검색했는가(검색 재현)를 채점

써야 하는 도구 (target_tools)

질문에 맞는 도구를 골랐는가(도구 선택)를 채점

계속 자라는 시험지

운영 중 발견한 실패 사례를 문항으로 추가합니다. 골든셋은 회귀 테스트입니다.

문항 유형과 확인하려는 실패

유형	확인하려는 실패
단일 조항 · 표 데이터	검색 품질, 표 수치 왜곡
약어 · 용어 불일치	질문과 문서의 용어가 다를 때
다중 홉 · 여러 소스 결합	도구 선택, 앞 결과를 이용한 다음 검색
같은 소스 여러 번 검색	질문 나누기, 불필요한 반복 호출
존재하지 않는 정보	환각 (없는 내용을 지어내는가)

TWO-LAYER SCORING

2계층 채점: 과정과 결과를 따로 본다

계층	지표	무엇과 비교	의미	가중치
과정 (40%)	도구 선택	써야 하는 도구	질문에 맞는 도구를 골랐는가	1/4
	검색어 품질	질문	검색어가 핵심을 담았는가	1/4
	호출 효율	퀘적	같은 정보를 반복해 찾지 않았는가	1/4
	결과 반영	퀘적 ↔ 답변	도구 결과를 보고 대응했는가	1/4
결과 (60%)	정답 일치	정답	핵심 사실(수치 · 조항 · 결론)이 정답과 같은가	0.4
	근거 충실	검색 결과	답변이 검색 결과에 근거하는가 (환각 여부)	0.2
	검색 재현	정답 ↔ 검색 결과	필요한 사실을 빠짐없이 가져왔는가	0.2
	답변 관련성	질문	질문에 직접 답하는가	0.1
	검색 정밀도	검색 결과	가져온 결과 중 쓸모 있는 비율	0.1

정답 일치의 비중이 가장 큰 이유: **엉뚱한 문서를 근거로 충실하게 쓴 답**은 근거 충실 · 관련성 점수가 높아도 틀린 답입니다. 정답과 비교해야만 잡을 수 있습니다.

LLM AS A JUDGE

채점기를 설계할 때 확인할 것

채점기는 다른 계열의 모델로

같은 모델이 자기 답을 채점하면 점수가 후해지는 경향(self-preference bias)이 있습니다. 문항 수만큼 호출되므로 **충분히 똑똑하면서 저렴한 모델**을 고릅니다.

과정 채점기와 결과 채점기의 역할을 나눈다

과정 채점기는 도구·검색어·순서를 보고 효율을 판단하고, 결과 채점기는 정답·검색 결과 전문·답변을 보고 사실을 판단합니다. 보지 못하는 것은 판단하지 않게 합니다.

판단에 필요한 정보를 실제로 받는가

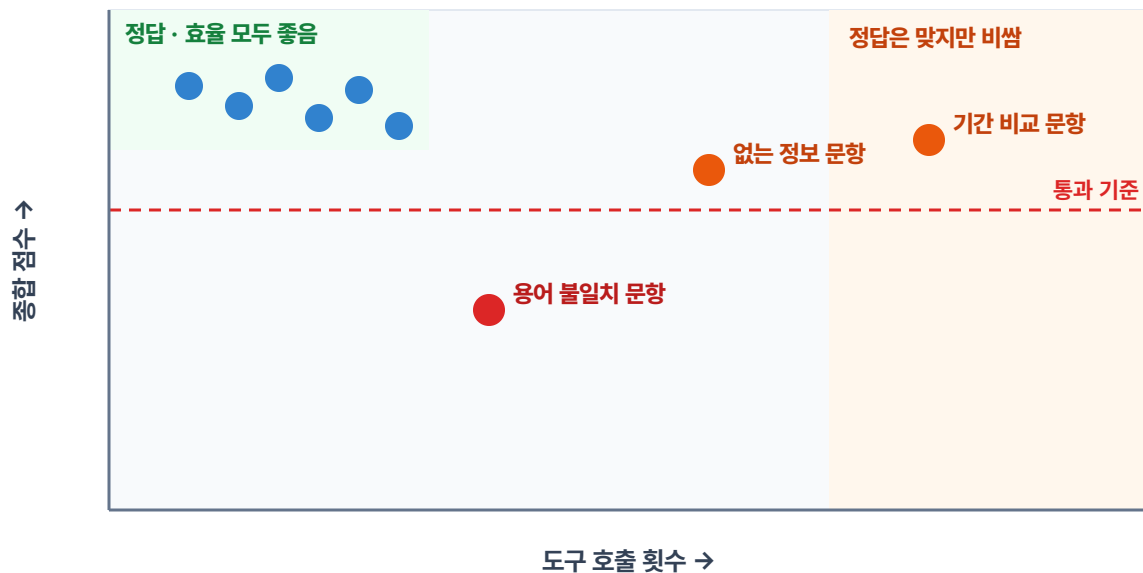
도구 결과의 앞부분만 잘라 보여 줬더니 문맥 헤더(문서명·목차)만 보여서, 채점기가 "근거 없이 답했다"며 정상 문항을 깎은 사례가 있습니다.

평균은 코드가 계산한다

각 지표는 LLM이 채점하고, 가중 평균은 코드가 계산합니다. 평균까지 LLM에 맡기면 계산이 틀리거나 가중치가 실행마다 달라집니다.

BASELINE

점수만 보면 놓치는 것: 점수와 비용을 함께 본다



평균과 통과율만 보면

대부분 합격이라 "잘 동작한다"고 판단하기 쉽습니다.

오른쪽으로 떨어진 점

정답은 맞췄지만 같은 검색을 반복해, 응답 시간과 비용이 다른 문항의 몇 배입니다.

아래쪽으로 떨어진 점

기준 미달 문항입니다. 어떤 지표가 낮은지 함께 봅니다.

LLM 실행이라 수치는 매번 조금씩 달라집니다. **한 번의 점수보다 패턴**을 봅니다.

DIAGNOSIS

진단: 증상에서 원인 계층으로

증상 (기준 미달 지표)	의심할 계층	수정 후보
엉뚱한 도구를 고름	도구 설명	도구 이름 · 설명(Docstring)을 구체적으로
맞는 도구인데 근거를 못 찾음	검색 · 인덱스	청킹, 메타데이터, 검색 방식, 가져오는 개수
같은 검색을 반복함	도구 인자 · 반환 형식	필터 인자 추가, 결과에 출처 · 위치 표기
질문 용어가 문서와 다름	질의 정규화	약어 사전, 답변 원칙 보강, 사용자에게 되묻기
없는 내용을 지어냄	답변 원칙	프롬프트의 근거 원칙, 필요하면 가드레일

미달 지표는 모두 표시한다

실패는 여러 요인이 겹쳐 생깁니다. 한 가지 유형으로 단정하지 않습니다.

"먼저 볼 곳" ≠ "주요 원인"

도구 선택 → 검색 → 생성 순서로 보는 것은 살펴볼 순서일 뿐입니다.

CASE STUDY

사례: 궤적에서 원인을 찾는다

질문: "1분기와 3분기의 반도체 매출을 비교해 줘" · 분기별 보고서 4개는 문장 구조가 거의 같음

도구 호출 궤적

Step 1	search_reports("1분기 반도체 매출")	1분기 보고서
Step 2	search_reports("3분기 반도체 매출")	2분기 보고서
Step 3	search_reports("2024년 3분기 반도체")	2분기 보고서
Step 4	search_reports("3Q 반도체 실적")	4분기 보고서
...	검색어를 바꿔 가며 반복	결국 3분기 확보

정답은 맞췄지만, 호출 횟수와 응답 시간이 다른 문항의 몇 배입니다.

원인 후보 판단

후보	판단
프롬프트 · 도구 설명	원인 아님 · 지침대로 분기별로 나눠 검색함
인덱스	원인 아님 · 청크마다 분기 메타데이터가 이미 있음
도구 인터페이스	원인 · 분기를 지정할 인자가 없어 메타데이터를 쓸 수 없음

에이전트가 아무리 다시 검색해도 고칠 수 없는 결함입니다. **도구에 필터가 없기 때문**입니다.

FIX

수정: 인덱스의 메타데이터를 도구 인자로 연다

BEFORE

```
@tool
def search_reports(query: str) -> str:
    """분기별 보고서를 검색합니다.
    분기 비교는 분기별로 나누어 호출하세요."""
    docs = store.similarity_search(query, k=2)
    return format(docs)    # 본문만 반환
```

AFTER

```
@tool
def search_reports(query: str,
                   quarter: int | None = None) -> str:
    """분기별 보고서를 검색합니다.
    분기 비교는 분기별로 나누어 호출하세요.
    특정 분기 수치가 필요하면 quarter를 지정하세요."""
    filt = {"quarter": quarter} if quarter else None
    docs = store.similarity_search(query, k=3, filter=filt)
    return format(docs)    # [분기 · 위치] + 본문
```

결과

분기마다 한 번씩만 검색하고 끝납니다. 호출 횟수와
응답 시간이 크게 줄고 정답 일치도 올라갑니다.

고친 곳

프롬프트도 모델도 아닌 **도구 인터페이스**입니다. 인덱싱
때 붙인 메타데이터가 인자로 열리면서 쓸모가
생겼습니다.

반환 형식도 함께

결과에 분기와 위치를 표기해, 에이전트가 받은 내용의
출처를 바로 알 수 있게 합니다.

REGRESSION CHECK

한 문항이 좋아졌다고 끝이 아니다: 회귀 확인

전체를 다시 채점한다

도구 설명이나 반환 형식을 바꾸면 **다른 문항의 행동도 달라질 수 있습니다**. 고친 문항만이 아니라 골든셋 전체를 다시 돌려 이전 결과와 비교합니다.

문항별 before · after · 차이(delta) · 판정(개선 / 회귀 / 변동 범위)

흔들림과 진짜 변화를 구분한다

- 차이가 ± 0.05 안쪽이면 LLM 실행과 채점의 흔들림으로 봅니다
- 표시가 붙은 문항은 먼저 **"이번에 바꾼 도구를 썼는가?"**를 확인합니다
- 관련 없는 문항이 크게 변했다면 2~3번 다시 돌려 평균을 봅니다
- 그래도 다르면 궤적을 열어 실제로 행동이 바뀌었는지 봅니다

회귀를 놓치는 것만큼 흔한 실수가 **변경과 무관한 개선을 이번 수정의 성과로 착각하는 것**입니다.

HUMAN IN THE LOOP

실행 중 사람이 개입하는 지점

검색만 하는 RAG는 잘못돼도 틀린 답변 하나로 끝납니다. 에이전트가 스스로 도구를 고르고 반복 호출하게 되면서, 비용과 위험을 통제할 개입 지점이 생겼습니다.

개입 지점	왜 필요한가	예
비싸거나 위험한 도구 호출 전 승인	자율성에는 비용 · 위험 통제가 따라야 함	전체 요약 검색, 실제 DB에 SQL 실행, 민감 문서 접근
답변 확정 전 검토	틀린 답의 대가가 큰 도메인	근거 점수가 낮은 답변을 담당자 검토 대기열로
질문 명확화	추측하며 반복 검색하는 것보다 되묻는 것이 싸고 정확	규정에 없는 직급 · 용어가 나오면 확인 질문
피드백 → 평가 루프	사람의 판정이 골든셋으로 쌓여야 평가가 자람	운영 중 실패 사례를 문항으로 추가

APPROVAL GATE

승인 게이트: 멈추고, 저장하고, 결정을 받아 재개

1

도구 호출 제안

에이전트가 비싼 도구를 호출하려 함



2

멈춤 (interrupt)

정책에 걸린 도구면 실행 직전에 멈추고
상태를 체크포인트에 저장



3

사람의 결정

approve 그대로 실행
edit 인자를 고쳐 실행
reject 사유와 대안 전달



4

재개 (resume)

저장된 상태에서 이어서 실행. 거절이면
사유를 읽고 다른 경로로 답을 찾음

```
agent = create_agent(  
    model=llm, tools=tools,  
    checkpoint=checkpoint, # 멈춘 상태를 저장  
    middleware=[HumanInTheLoopMiddleware(  
        interrupt_on={  
            "global_summary_search": {  
                "allowed_decisions": ["approve", "edit", "reject"]},  
        }],  
    )
```

도구 단위 정책이라 에이전트 코드나 프롬프트를 고치지 않고 붙였다 떼 수
있습니다.

어떤 도구에 게이트를 둘지는 평가 리포트의 도구별 호출 수와 소요 시간을 근거로
정합니다.

IN PRODUCTION

평가를 운영에 배치하기

개발 중

진단과 개선 확인

평가 러너로 문항별 점수 · 궤적을 보고 고친 뒤
비교합니다.

목적: 원인 찾기

배포 전

CI 품질 게이트

코드 · 프롬프트가 바뀌면 골든셋 전체를 채점해,
평균이 기준 이상이고 회귀가 없을 때만 배포합니다.

목적: 회귀 차단

운영 중

트래픽 샘플 채점

응답은 바로 돌려주고, 로그의 5~10%만 비동기로
채점해 대시보드(LangSmith 등)에 올립니다.

목적: 골든셋에 없던 실패 발견

↪ 운영 중 발견한 실패 → 골든셋에 문항 추가 → 다음 배포의 품질 게이트에 포함

평가는 사용자 응답을 느리게 하지 않습니다. **실시간 응답과 채점을 분리**하고, 전수 대신 표본만 채점해 비용을 관리합니다.

KEY ELEMENTS

4교시 정리: 평가로 시스템을 개선하는 루프

단계	핵심	설계 질문
평가	실패 방식마다 문항, 과정 + 결과 2계층 채점	정답뿐 아니라 과정과 비용도 보고 있는가?
진단	미달 지표 + 궤적 → 원인 계층(도구 · 인덱스 · 프롬프트)	에이전트가 스스로 못 고치는 결함은 무엇인가?
수정	원인이 있는 계층을 고침 (예: 메타데이터를 도구 인자로)	프롬프트로 덮지 않고 원인을 고쳤는가?
재평가	전체 재채점, 흔들림과 진짜 변화 구분	다른 문항이 나빠지지 않았는가?
실행 중 개입	비싸거나 위험한 도구 앞의 승인 게이트	게이트를 둘 근거(호출 수 · 시간)가 있는가?

COURSE WRAP-UP

과정 전체 정리

1교시

Agentic RAG 개요

검색을 파이프라인의 한 단계가 아니라 에이전트가 고르는 도구로. 루프는 짧고, 품질은 도구와 데이터에서 결정됨

2교시

엔터프라이즈 데이터 인덱싱

정규화 → 문맥 헤더 → 하이브리드 검색 → 지식 그래프 · GraphRAG. 데이터마다 맞는 인덱스를 만들고 도구로 노출

3교시

MCP & Skills 연동

소스별 검색 도구 설계, MCP 서버 분리와 인증 · 인가, 도구가 많아지면 Skills로 필요한 만큼만

4교시

평가 기반 자가 개선

골든셋과 2계층 채점으로 진단하고, 원인 계층을 고친 뒤 회귀까지 확인. 비싼 도구 앞에는 승인 게이트

도구와 데이터를 설계하고, **평가로 개선을 증명하는 것**. 이것이 Agentic RAG를 운영 가능한 시스템으로 만드는 일입니다.